

Ejercicios 6.10 - Estructuras de datos en Julia

Heiber Alexander Pantevez Villamil

Tabla de contenidos

1	Introducción	1
2	Ejercicio 1: Rutas de monitoreo (tuplas y vectores)	2
2.1	Análisis técnico del Ejercicio 1	2
3	Ejercicio 2: Metadatos y ecosistemas (diccionarios y conjuntos)	3
3.1	Análisis técnico del Ejercicio 2	3
4	Pregunta general: slicing o extracción por rangos	4
5	Conclusiones	4
6	Archivos entregables	5
7	Renderización sugerida	5

1 Introducción

En esta entrega se desarrollan los ejercicios 6.10 sobre estructuras de datos en **Julia**, aplicados a un contexto geomático y de monitoreo ecológico. La solución utiliza **tuplas** para representar coordenadas fijas, **vectores** para secuencias mutables, **diccionarios** para almacenar atributos mediante pares clave-valor y **conjuntos** para depurar duplicados e identificar coincidencias mediante operaciones lógicas.

El documento presenta los bloques de código en Julia y responde argumentativamente las preguntas analíticas solicitadas en la guía.

2 Ejercicio 1: Rutas de monitoreo (tuplas y vectores)

```
# 1) Tuplas estáticas con coordenadas
punto_1 = (11.1198, -74.0321)
punto_2 = (11.1250, -74.0280)
punto_3 = (11.1302, -74.0215)

# 2) Colección secuencial y mutable
ruta_avistamiento = [punto_1, punto_2, punto_3]

# 3) Agregar nuevo punto al final
punto_4 = (11.1355, -74.0150)
push!(ruta_avistamiento, punto_4)

# 4) Extraer último punto usando indexación dinámica
ultimo_reporte = ruta_avistamiento[end]

# 5) Vector paralelo de elevaciones
elevaciones = [850.5, 920.0, 1050.2, 1180.8]

# 6) Cálculos solicitados
elevacion_maxima = maximum(elevaciones)
total_puntos = length(ruta_avistamiento)

println("Ruta completa: ", ruta_avistamiento)
println("Último reporte: ", ultimo_reporte)
println("Elevación máxima: ", elevacion_maxima, " msnm")
println("Número total de puntos visitados: ", total_puntos)
```

2.1 Análisis técnico del Ejercicio 1

La ventaja de guardar cada coordenada individual como una **tupla inmutable** radica en la protección de la integridad espacial del dato. Una coordenada representa una unidad lógica compuesta por latitud y longitud; por tanto, no conviene que uno de sus componentes pueda alterarse accidentalmente mientras el otro permanece igual. En términos de topología y consistencia geomática, esta decisión reduce el riesgo de introducir errores silenciosos en la posición de los puntos.

En cambio, la **ruta completa** debe almacenarse en una estructura **mutable** porque el recorrido sí cambia con el tiempo. En una campaña de monitoreo es normal añadir nuevos

puntos, intercalar reportes o ampliar la secuencia. Por eso, la combinación correcta en Julia es usar **tuplas para los puntos** y un **vector para la ruta**.

Si se intenta modificar directamente la latitud de `punto_1`, por ejemplo con una instrucción como `punto_1[1] = 12.0000`, Julia generará un error, ya que las tuplas no permiten reasignación de elementos. Ese comportamiento es deseable porque obliga a hacer cualquier cambio de forma explícita y controlada.

3 Ejercicio 2: Metadatos y ecosistemas (diccionarios y conjuntos)

```
# 1) Diccionario del parque
pnn_tayrona = Dict{
    "nombre" => "PNN Tayrona",
    "area_hectareas" => 15000,
    "abierto_turismo" => true,
    "ecosistemas" => ["Manglar", "Bosque Seco", "Coral"]
}

# 2) Acceso seguro a clave inexistente
fecha_creacion = get(pnn_tayrona, "fecha_creacion", "Dato no disponible")
println("Fecha de creación: ", fecha_creacion)

# 3) Actualizar diccionario
pnn_tayrona["departamento"] = "Magdalena"

# 4) Observaciones crudas con duplicados
observaciones_crudas = ["Manglar", "Coral", "Bosque Seco", "Manglar", "Coral", "Matorral"]
observaciones_unicas = Set(observaciones_crudas)

# 5) Intersección entre ecosistemas oficiales y observaciones únicas
ecosistemas_oficiales = Set(pnn_tayrona["ecosistemas"])
ecosistemas_avistados = intersect(ecosistemas_oficiales, observaciones_unicas)

println("Observaciones únicas: ", observaciones_unicas)
println("Ecosistemas oficiales avistados hoy: ", ecosistemas_avistados)
```

3.1 Análisis técnico del Ejercicio 2

Si este mismo problema se resolviera en **R**, para almacenar la colección de ecosistemas dentro de las propiedades del parque lo más adecuado sería usar `c("Manglar", "Bosque Seco",`

"Coral"), pero dentro de una **lista nombrada** que represente el parque completo. La razón es que los ecosistemas son datos homogéneos del mismo tipo (texto), y `c()` crea justamente un **vector atómico de caracteres**, que es la forma más simple y apropiada para una colección plana de cadenas.

En consecuencia, una estructura razonable en R sería algo como:

```
#| eval: false
pnn_tayrona <- list(
  nombre = "PNN Tayrona",
  area_hectareas = 15000,
  abierto_turismo = TRUE,
  ecosistemas = c("Manglar", "Bosque Seco", "Coral")
)
```

Se usaría `list()` para agrupar los atributos del parque, pero `c()` para construir específicamente el vector de ecosistemas, porque no se requiere mezclar tipos dentro de esa subestructura.

4 Pregunta general: slicing o extracción por rangos

La diferencia crítica entre ejecutar `ruta[1:3]` en Python y ejecutar el mismo código en R o Julia está en dos reglas simultáneas: el **índice de origen** y la **inclusión o exclusión del extremo final**.

En **Python**, la indexación empieza en **0** y el rango final es **exclusivo**. Por tanto, `ruta[1:3]` devuelve los elementos ubicados en los índices 1 y 2, es decir, **dos elementos**.

En **R** y **Julia**, la indexación empieza en **1** y el rango es **inclusivo** en ambos extremos. Por tanto, `ruta[1:3]` devuelve los elementos 1, 2 y 3, es decir, **tres elementos**.

Esta diferencia es una fuente clásica de errores cuando se migra código entre lenguajes, porque una misma expresión escrita literalmente igual no produce la misma porción de datos.

5 Conclusiones

La resolución de estos ejercicios muestra que la selección de la estructura de datos no es una decisión meramente sintáctica, sino una decisión técnica con implicaciones en seguridad, legibilidad y robustez del análisis. En Julia, las tuplas son apropiadas para fijar coordenadas, los vectores para administrar secuencias, los diccionarios para modelar atributos tipo GeoJSON y los conjuntos para limpiar duplicados y comparar colecciones mediante lógica de conjuntos.

6 Archivos entregables

La carpeta del ejercicio debe incluir, como mínimo:

- El script fuente en Julia (.jl).
- El documento analítico en Quarto (.qmd).
- Los renders finales en HTML y PDF.

7 Renderización sugerida

```
#| eval: false
quarto render ejercicios_6_10_corregido.qmd --to html
quarto render ejercicios_6_10_corregido.qmd --to pdf
```